

9 Type Casting in python

conversion of one data type to another data type is nothing but type casting

There are two types of data conversions

1. Internal Type casting (Coercion) Python choose to select the data type automatically for the result where there is a minimal data loss.
2. External Type casting

```
▶ # Internal Type Conversion
a = 10 # integer
b = 3.5 # float
result = a + b # automatic type conversion to float
print("\nInternal Type Conversion:")
print("a:", type(a))
print("b:", type(b))
print("Result of a + b:", result)
```

[2] ✓ 0.0s

...

```
Internal Type Conversion:
a: <class 'int'>
b: <class 'float'>
Result of a + b: 13.5
```



```
# Data type casting in Python
x = 5 # integer
y = 2.5 # float
z = "10" # string
print("Original data types:")
print("x:", type(x))
print("y:", type(y))
print("z:", type(z))
print("\nCasting data types:")
print(x + y + int(z)) # casting string to integer
print(type(x + y + int(z)))
```

[4] ✓ 0.0s

```
... Original data types:
x: <class 'int'>
y: <class 'float'>
z: <class 'str'>
```

```
Casting data types:
17.5
<class 'float'>
```



```
# convert the values which are compatible only
print(int('10a')) # This will raise a ValueError because '10a' cannot be converted to an integer
```

[5] ✗ 0.0s

```
... -----
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_53555/3922154625.py in <cell line: 0>()
      1 # convert the values which are compatible only
----> 2 print(int('10a')) # This will raise a ValueError because '10a' cannot be converted to an integer

ValueError: invalid literal for int() with base 10: '10a'
```

```
# list to tuple conversion
my_list = [1, 2, 3, 4, 5]
my_tuple = tuple(my_list)
print("Original list:", my_list)
print("Converted tuple:", my_tuple)
```

[9] ✓ 0.0s

```
... Original list: [1, 2, 3, 4, 5]
    Converted tuple: (1, 2, 3, 4, 5)
```

▷ ▾

```
#tuple to list conversion
my_tuple = (1, 2, 3, 4, 5)
my_list = list(my_tuple)
print("Original tuple:", my_tuple)
print("Converted list:", my_list)
```

[10] ✓ 0.0s

```
... Original tuple: (1, 2, 3, 4, 5)
    Converted list: [1, 2, 3, 4, 5]
```

▷ ▾

```
a=1
b=2
c=complex(a,b)
print("Complex number:", c)
print("Real part:", c.real)
print(type(c.real))
print("Imaginary part:", c.imag)
print(type(c.imag))
print("Type of complex number:", type(c))
```

[3] ✓ 0.0s

```
... Complex number: (1+2j)
    Real part: 1.0
    <class 'float'>
    Imaginary part: 2.0
    <class 'float'>
    Type of complex number: <class 'complex'>
```

```
# Creating a dict using zip from nested tuples
tup = ((1, 'a'), (2, 'b'), (3, 'c'), (4, 'd'), (5, 'e'))
print(dict(tup))
```

[6] ✓ 0.0s

... {1: 'a', 2: 'b', 3: 'c', 4: 'd', 5: 'e'}

▷ ✓

```
value=1
print(bool(value))
value1=0
print(bool(value1))
```

[8] ✓ 0.0s

... True
False